

```
In [1]: # Import Library

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
In [2]: # Loading dataset

df = pd.read_csv(r'C:\Users\user\FSDS_Adv_Gene_AI_Agentic_AI\Machine Learning\Regression\SLR_
df
```

```
Out[2]:
```

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19
...	...	...
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

200 rows × 2 columns

```
In [3]: df.head()
```

```
Out[3]:
```

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19

```
In [4]: df.tail()
```

```
Out[4]:
```

	study_hours	student_marks
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

```
In [5]: df.shape
```

```
Out[5]: (200, 2)
```

```
In [6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   study_hours      195 non-null   float64
1   student_marks    200 non-null   float64
dtypes: float64(2)
memory usage: 3.3 KB
```

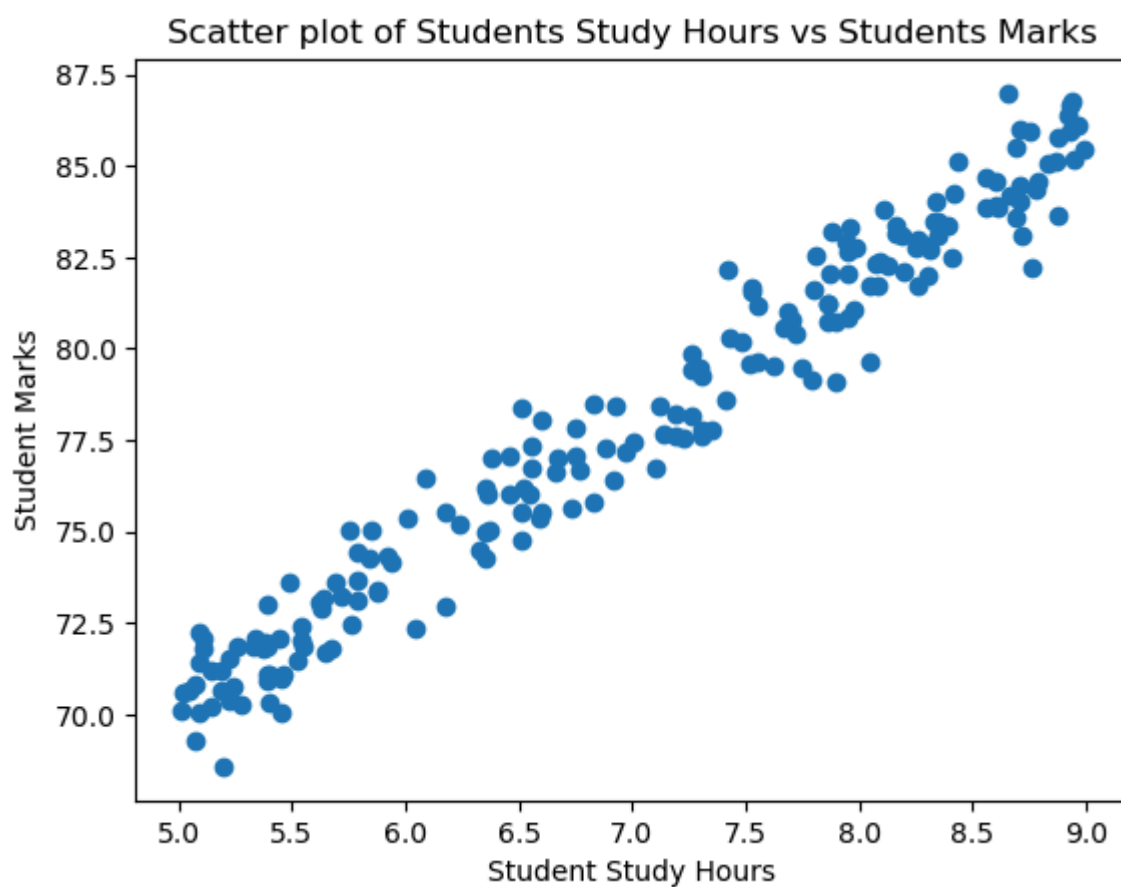
```
In [7]: # Descriptive Statistics
```

```
df.describe()
```

```
Out[7]:
```

	study_hours	student_marks
count	195.000000	200.000000
mean	6.995949	77.93375
std	1.253060	4.92570
min	5.010000	68.57000
25%	5.775000	73.38500
50%	7.120000	77.71000
75%	8.085000	82.32000
max	8.990000	86.99000

```
In [10]: plt.scatter(x = df.study_hours, y = df.student_marks)
plt.xlabel('Student Study Hours')
plt.ylabel('Student Marks')
plt.title('Scatter plot of Students Study Hours vs Students Marks')
plt.show()
```



Prepare the data for Machine Learning Algorithm

Data Cleaning

In [11]: df

Out[11]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19
...	...	...
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

200 rows × 2 columns

In []: # Checking missing values

```
In [12]: df.isnull().sum()
```

```
Out[12]: study_hours      5
student_marks      0
dtype: int64
```

```
In [13]: df.mean()
```

```
Out[13]: study_hours      6.995949
student_marks      77.933750
dtype: float64
```

```
In [18]: df2 = df.fillna(df.mean())
df2.head()
```

```
Out[18]:
```

	study_hours	student_marks
0	6.830000	78.50
1	6.560000	76.74
2	6.995949	78.68
3	5.670000	71.82
4	8.670000	84.19

```
In [19]: df.head()
```

```
Out[19]:
```

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19

```
In [20]: df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   study_hours     200 non-null   float64
1   student_marks   200 non-null   float64
dtypes: float64(2)
memory usage: 3.3 KB
```

```
In [21]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   study_hours     195 non-null   float64
1   student_marks   200 non-null   float64
dtypes: float64(2)
memory usage: 3.3 KB
```

Splitting the dataset

```
In [22]: x = df2.drop('student_marks', axis = 'columns')
y = df2.drop('study_hours', axis = 'columns')

print('Shape of x', x.shape)
print('Shape of y', y.shape)
```

Shape of x (200, 1)

Shape of y (200, 1)

```
In [25]: x
```

```
Out[25]:
```

	study_hours
--	-------------

0	6.830000
---	----------

1	6.560000
---	----------

2	6.995949
---	----------

3	5.670000
---	----------

4	8.670000
---	----------

...	...
-----	-----

195	7.530000
-----	----------

196	8.560000
-----	----------

197	8.940000
-----	----------

198	6.600000
-----	----------

199	8.350000
-----	----------

200 rows × 1 columns

```
In [26]: y
```

```
Out[26]:
```

	student_marks
--	---------------

0	78.50
---	-------

1	76.74
---	-------

2	78.68
---	-------

3	71.82
---	-------

4	84.19
---	-------

...	...
-----	-----

195	81.67
-----	-------

196	84.68
-----	-------

197	86.75
-----	-------

198	78.05
-----	-------

199	83.50
-----	-------

200 rows × 1 columns

```
In [27]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 0.20, random_state = 0)

print('Shape of x_train', x_train.shape)
print('Shape of x_test', x_test.shape)
print('Shape of y_train', y_train.shape)
print('Shape of y_test', y_test.shape)
```

```
Shape of x_train (160, 1)
Shape of x_test (40, 1)
Shape of y_train (160, 1)
Shape of y_test (40, 1)
```

```
In [28]: x_train
```

```
Out[28]:
```

	study_hours
134	6.51
66	7.86
26	6.51
113	7.95
168	7.95
...	...
67	8.26
192	8.71
117	8.83
47	5.01
172	7.35

160 rows × 1 columns

```
In [29]: x_test.shape
```

```
Out[29]: (40, 1)
```

```
In [30]: y_train
```

Out[30]:

student_marks	
134	78.39
66	81.25
26	74.75
113	80.86
168	82.68
...	...
67	81.70
192	84.03
117	85.04
47	70.11
172	77.78

160 rows × 1 columns

In [32]:

y_test

Out[32]:

student_marks	
18	82.50
170	71.18
107	73.25
98	83.64
177	73.64
182	86.99
5	81.18
146	82.75
12	79.50
152	81.70
61	79.41
125	85.95
180	77.19
154	78.45
80	84.00
7	85.46
33	84.35
130	73.19
37	78.21
74	77.59
183	83.87
145	85.15
45	72.96
159	80.72
60	73.61
123	79.53
179	78.17
185	79.63
122	76.83
44	82.38
16	76.04
55	85.48
150	71.87
111	75.04
22	70.67
189	79.87

student_marks	
129	74.49
4	84.19
83	75.36
106	72.10

Select a model and train

```
In [34]: #  $y = mx + c$ 

from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

```
In [35]: lr
```

```
Out[35]: ▼ LinearRegression ⓘ ?
        ► Parameters
```

```
In [36]: lr.fit(x_train,y_train)
```

```
Out[36]: ▼ LinearRegression ⓘ ?
        ► Parameters
```

```
In [37]: lr.get_params()
```

```
Out[37]: {'copy_X': True,
          'fit_intercept': True,
          'n_jobs': None,
          'positive': False,
          'tol': 1e-06}
```

```
In [38]: lr.coef_ # m
```

```
Out[38]: array([[3.93037294]])
```

```
In [39]: lr.intercept_ # c
```

```
Out[39]: array([50.45063632])
```

```
In [41]: m = 3.93
c = 50.45
#  $y = mx + c$ 
y = m * 10 + c
y
```

```
Out[41]: 89.75
```

```
In [42]: m = 3.93
c = 50.45
#  $y = mx + c$ 
y = m * 11 + c
y
```

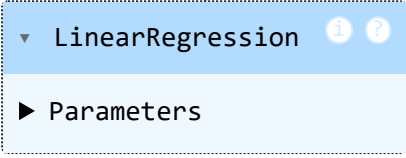
Out[42]: 93.68

```
In [60]: lr.predict([[11]]).round(2)
```

C:\Users\user\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(

Out[60]: array([[93.68]])

```
In [45]: lr
```

Out[45]:  LinearRegression ⓘ ?

► Parameters

lr.predict([[5]])[0][0].round(2)

```
In [46]: y_pred = lr.predict(x_test)
y_pred
```

Out[46]: array([[83.50507271],
[70.84927186],
[72.93236952],
[85.35234799],
[73.20749562],
[84.48766595],
[80.12495199],
[81.85431608],
[80.91102657],
[82.20804964],
[78.98514384],
[84.84139951],
[77.84533568],
[77.68812077],
[83.22994661],
[85.78468901],
[84.9593107],
[72.61793968],
[78.71001773],
[79.18166248],
[84.2911473],
[85.6274741],
[74.74034107],
[81.3433676],
[72.02838374],
[80.40007809],
[78.98514384],
[82.09013845],
[77.94732382],
[82.24735337],
[75.44780819],
[84.60557713],
[71.63534645],
[75.48711192],
[70.29901965],
[78.98514384],
[75.32989701],
[84.52696967],
[74.07217767],
[71.4388278]])

```
In [47]: pd.DataFrame(np.c_[x_test, y_test, y_pred], columns = ['study_ hours', 'students_marks_origin
```

Out[47]:

	study_hours	students_marks_original	student_marks_prediction
0	8.410000	82.50	83.505073
1	5.190000	71.18	70.849272
2	5.720000	73.25	72.932370
3	8.880000	83.64	85.352348
4	5.790000	73.64	73.207496
5	8.660000	86.99	84.487666
6	7.550000	81.18	80.124952
7	7.990000	82.75	81.854316
8	7.750000	79.50	80.911027
9	8.080000	81.70	82.208050
10	7.260000	79.41	78.985144
11	8.750000	85.95	84.841400
12	6.970000	77.19	77.845336
13	6.930000	78.45	77.688121
14	8.340000	84.00	83.229947
15	8.990000	85.46	85.784689
16	8.780000	84.35	84.959311
17	5.640000	73.19	72.617940
18	7.190000	78.21	78.710018
19	7.310000	77.59	79.181662
20	8.610000	83.87	84.291147
21	8.950000	85.15	85.627474
22	6.180000	72.96	74.740341
23	7.860000	80.72	81.343368
24	5.490000	73.61	72.028384
25	7.620000	79.53	80.400078
26	7.260000	78.17	78.985144
27	8.050000	79.63	82.090138
28	6.995949	76.83	77.947324
29	8.090000	82.38	82.247353
30	6.360000	76.04	75.447808
31	8.690000	85.48	84.605577
32	5.390000	71.87	71.635346
33	6.370000	75.04	75.487112
34	5.050000	70.67	70.299020
35	7.260000	79.87	78.985144

	study_hours	students_marks_original	student_marks_prediction
36	6.330000	74.49	75.329897
37	8.670000	84.19	84.526970
38	6.010000	75.36	74.072178
39	5.340000	72.10	71.438828

```
In [ ]: # Fine Tune your Model
```

```
In [48]: lr
```

```
Out[48]: LinearRegression
```

Parameters

```
In [49]: lr.score(x_test, y_test) # Variance
```

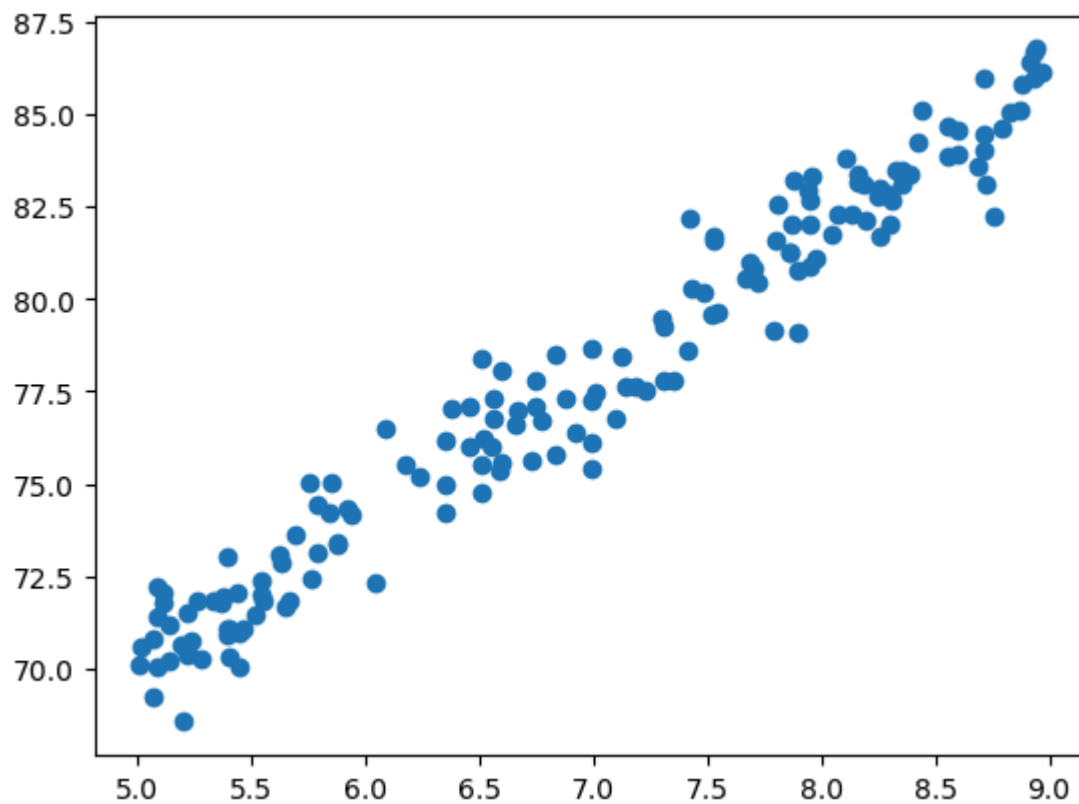
```
Out[49]: 0.9521841793508594
```

```
In [51]: lr.score(x_train, y_train) # bias
```

```
Out[51]: 0.9584528455152638
```

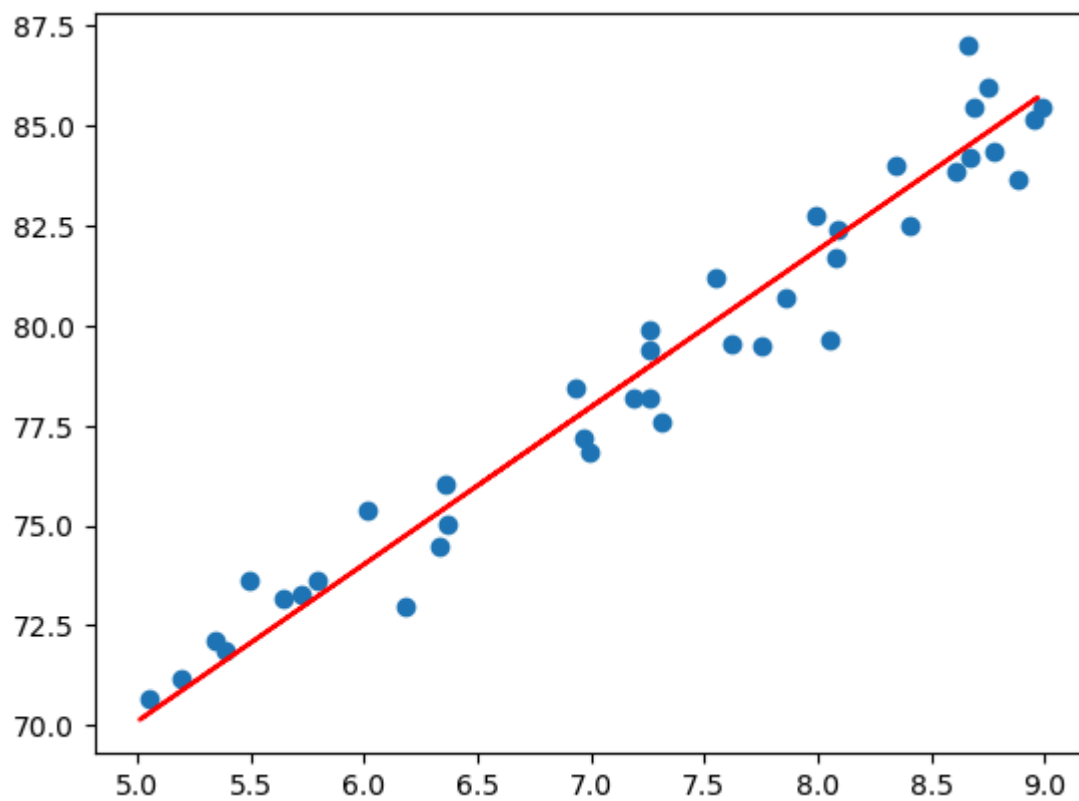
```
In [52]: plt.scatter(x_train, y_train)
```

```
Out[52]: <matplotlib.collections.PathCollection at 0x218a280cb90>
```



```
In [53]: plt.scatter(x_test, y_test)
plt.plot(x_train, lr.predict(x_train), color = 'r')
```

```
Out[53]: [<matplotlib.lines.Line2D at 0x218a2835d10>]
```



```
In [ ]: # Present your solution
```

```
In [ ]: # Save your Model
```

```
In [54]: lr
```

```
Out[54]: ▼ LinearRegression ⓘ ?
```

```
► Parameters
```

```
In [55]: import joblib
joblib.dump(lr, 'Desktop.pkl')
```

```
Out[55]: ['Desktop.pkl']
```

```
In [56]: pwd
```

```
Out[56]: 'C:\\Users\\user\\FSDS_Adv_Gene_AI_&Agentic_AI\\Machine Learning\\Regression\\SLR_Models'
```

```
In [57]: model = joblib.load('Desktop.pkl')
```

```
In [58]: pwd
```

```
Out[58]: 'C:\\Users\\user\\FSDS_Adv_Gene_AI_&Agentic_AI\\Machine Learning\\Regression\\SLR_Models'
```

```
In [ ]: # lr.predict([[5]])[0][0].round(2)
```

```
In [ ]: # Launch, Monitor and Maintain your System
```

```
In [59]: __name__
```

```
Out[59]: '__main__'
```